

Міністерство освіти і науки України
Національний університет «Запорізька політехніка»

Кафедра програмних засобів

ЗВІТ

Дисципліна «Створення інтегрованих середовищ розробки»

Робота №1

Тема «Лексичний аналізатор»

Виконав варіант 19

Студент КНТ-122

Онищенко О.А.

Прийняли

Викладач

Дейнега Л.Ю.

2025

МЕТА

Метою роботи є ознайомитись з основами процесу виконання лексичного аналізу вихідної програми та інструментами, які використовуються під час реалізації даного процесу; навчитися проєктувати та розробляти лексичний і синтаксичний аналізатори у складі компілятора за допомогою генератора парсерів ANTLR.

ЗАВДАННЯ

Завдання на роботу:

1. Ознайомитись з теоретичними відомостями, необхідними для виконання роботи.
2. Визначити граматику для виконання лексичного аналізу програм мовою, заданою у відповідності з індивідуальним завданням з додатку А, узгодженим з викладачем.

Розробити IDE для мови програмування Rust. Обов'язково мають бути проаналізовані наступні можливості даної мови: вбудовані типи даних, функції, вектори, записи, методи, цикли, умовні оператори, зв'язування змінних, арифметичні оператори.

3. Побудувати кінцеві автомати для представлення сформованої граматики.

4. Побудувати лексичний аналізатор за допомогою генератора ANTLR мовою програмування Java:

- програма повинна виконувати лексичний аналіз вхідного тексту програми у відповідності з визначеним індивідуальним завданням;
- програма повинна обробляти і виводити повідомлення про всі помилки, виявлені під час лексичного аналізу з зазначенням причини помилки (недопустимі символи, пропущені лапки, тощо);

- програма повинна мати графічний інтерфейс та дозволяти задавати вхідний код програми для аналізу, задаючи його безпосередньо в інтерфейсі або створюючи файл програми, завантажуючи його через інтерфейс та виводячи результати лексичного аналізу.

5. Створити git-репозиторій для роботи над проєктом та додати в нього розроблений лексичний аналізатор.

6. Виконати тестування побудованого лексичного аналізатора.

7. Оформити звіт.

8. Відповісти на контрольні питання.

ВИКОНАННЯ

1 Лексична граматика

```
grammar Rust;

// --- PARSER ---
start: block | method | function | struct | if | else | if_else | else_if;

// Block
block: LEFT_CURLY_BRACE .*? RIGHT_CURLY_BRACE;
method: FN ID LEFT_PARENTHESIS .*? RIGHT_PARENTHESIS RIGHT_ARROW? ID? block;
function: FN ID LEFT_PARENTHESIS .*? RIGHT_PARENTHESIS block;
struct: STRUCT ID LEFT_CURLY_BRACE .*? RIGHT_CURLY_BRACE | IMPL ID block;
if: IF .*? block;
else: ELSE .*? block;
if_else: if else;
else_if: ELSE IF .*? block;

// Inline
variable: LET MUT? ID COLON? (data_types)? EQ binding;
binding: FLOAT_LIT | INT_LIT | STRING_LIT | BOOL_LIT;
vector: VEC_MACRO LEFT_SQUARE_BRACE .*? RIGHT_SQUARE_BRACE;
loop: LOOP block | WHILE .*? block | FOR .*? block;

// Helper
data_types: INTEGER | UNSIGNED_INTEGER | FLOAT | BOOL | CHAR | STRING;

// --- LEXER ---
// Space and commentaries
WS: [ \t\r\n]+ -> skip;
SINGLE_COMMENT: '//' ~[\r\n]* -> skip;
DOUBLE_COMMENT: '/*' .*? '*/' -> skip;

// Key words
```

```

AS: 'as';
ASYNC: 'async';
AWAIT: 'await';
BREAK: 'break';
CONST: 'const';
CONTINUE: 'continue';
CRATE: 'crate';
DYN: 'dyn';
ELSE: 'else';
ENUM: 'enum';
EXTERN: 'extern';
FALSE: 'false';
FN: 'fn';
FOR: 'for';
IF: 'if';
IMPL: 'impl';
IN: 'in';
LET: 'let';
LOOP: 'loop';
MATCH: 'match';
MOD: 'mod';
MOVE: 'move';
MUT: 'mut';
PUB: 'pub';
REF: 'ref';
RETURN: 'return';
SELF: 'self';
STATIC: 'static';
STRUCT: 'struct';
SUPER: 'super';
TRAIT: 'trait';
TRUE: 'true';
TYPE: 'type';
USE: 'use';
WHERE: 'where';
WHILE: 'while';

// Literals and identifiers
BOOL_LIT: 'true' | 'false';
INT_LIT: [0-9]+;
FLOAT_LIT: [0-9]+ '.' [0-9]+ | '.' [0-9]+;
STRING_LIT: '"' (~["\\] | '\\' .)* '"';
ID: [a-zA-Z_] [a-zA-Z0-9_]*;

// Data types
INTEGER: 'i8' | 'i16' | 'i32' | 'i64' | 'i128' | 'isize';
UNSIGNED_INTEGER: 'u8' | 'u16' | 'u32' | 'u64' | 'u128' | 'usize';
FLOAT: 'f32' | 'f64';
BOOL: 'bool';
CHAR: 'char';
STRING: 'str' | 'String';

// Brackets
LEFT_CURLY_BRACE: '{';
RIGHT_CURLY_BRACE: '}';
LEFT_SQUARE_BRACE: '[';

```

```

RIGHT_SQUARE_BRACE: ']' ;
LEFT_PARENTHESIS: '(' ;
RIGHT_PARENTHESIS: ')' ;

// Symbols
EXCLAMATION: '!' ;
EXCLAMATORY_EQUAL: '!=' ;
PERCENT: '%' ;
AMPERSAND: '&' ;
AMPERSAND_EQUAL: '&=' ;
AMPERSAND_AMPERSAND: '&&' ;
STAR: '*' ;
STAR_EQUAL: '*=' ;
PLUS: '+' ;
PLUS_EQUAL: '+=' ;
COMMA: ',' ;
DASH: '-' ;
DASH_EQUAL: '-=' ;
RIGHT_ARROW: '->' ;
DOT: '.' ;
DOT_DOT: '..' ;
DOT_DOT_EQUAL: '..=' ;
DOT_DOT_DOT: '...' ;
SLASH: '/' ;
SLASH_EQUAL: '/=' ;
COLON: ':' ;
SEMICOLON: ';' ;
LEFT_LEFT: '<<' ;
LEFT_LEFT_EQUAL: '<<=' ;
LEFT: '<' ;
LEFT_EQUAL: '<=' ;
EQUAL: '=' ;
EQUAL_EQUAL: '==' ;
RIGHT_ARROW_BIG: '=>' ;
RIGHT: '>' ;
RIGHT_EQUAL: '>=' ;
RIGHT_RIGHT: '>>' ;
RIGHT_RIGHT_EQUAL: '>>=' ;
AT: '@' ;
CARET: '^' ;
CARET_EQUAL: '^=' ;
PIPE: '|' ;
PIPE_EQUAL: '|=' ;
PIPE_PIPE: '||' ;
QUESTION: '?' ;
UNDERSCORE: '_' ;
PATH_SEPARATOR: '::' ;
POUND: '#' ;
DOLLAR: '$' ;

// Vector macro
VEC_MACRO: 'vec!' ;

```

2 Кінцеві автомати, які використовуються під час аналізу

Кінцеві автомати наведено на рисунках нижче:

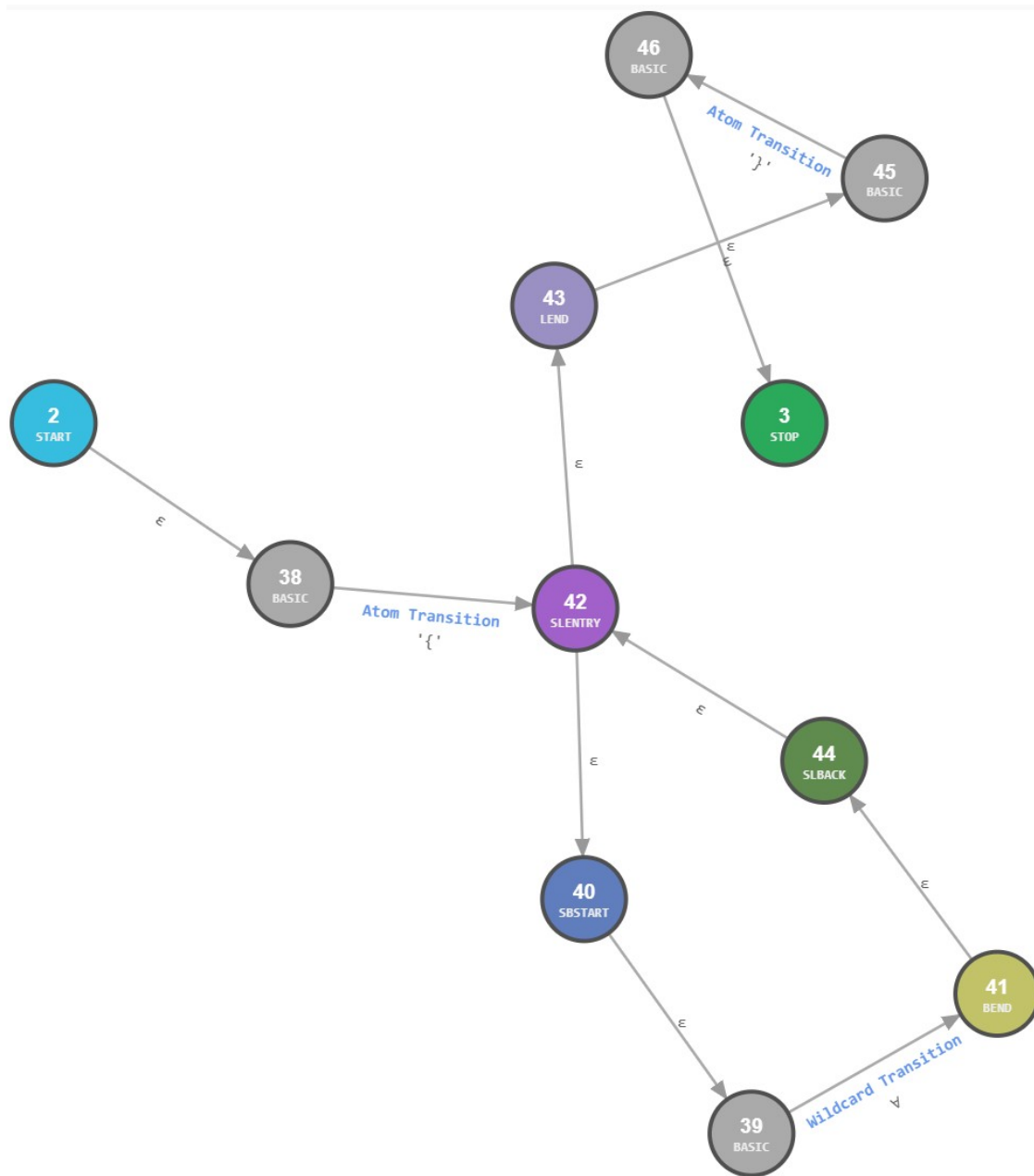


Рисунок 2.1 — Автомат *block*

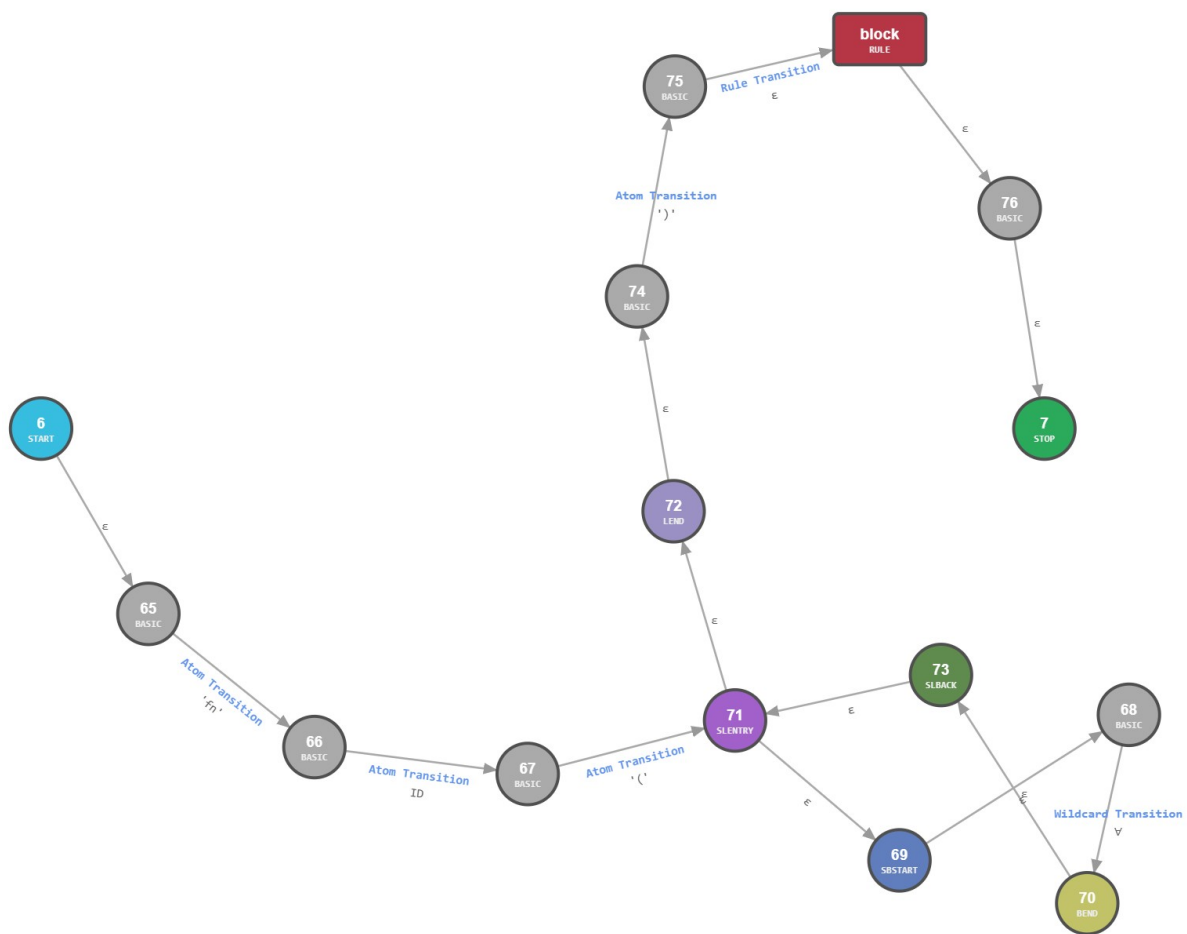


Рисунок 2.2 — Автомат *function*

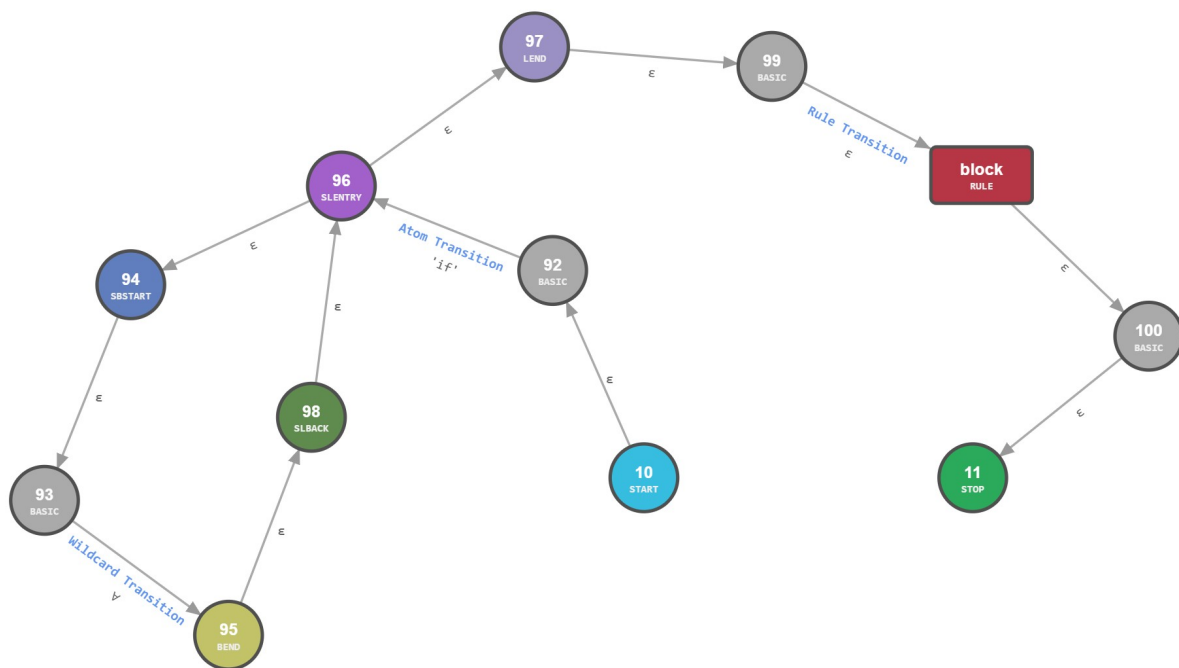


Рисунок 2.3 — Автомат *if*

Kod

```

import javax.swing.*;
import javax.swing.event.DocumentEvent;
import javax.swing.event.DocumentListener;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.io.*;

import gen.RustLexer;
import gen.RustParser;
import org.antlr.v4.runtime.*;
import org.antlr.v4.runtime.tree.*;

public class Main {
    public static String appName = "42SI.1 Text Editor";

    public static void main(String[] args) {
        String initialText = "fn main() {\n  let x = 10;\n  println!('Jesus is LORD')\n  let y = 30;\n\n  for (int i = 0; i <= 10; i++) {\n    println!('Jesus is KING', i)\n  }\n}";
        JFrame frame = new JFrame(appName);
        JTextArea inputField = new JTextArea(initialText);
        inputField.setFont(new Font("Consolas", Font.PLAIN, 14));
        JTextArea outputField = new JTextArea();
        JButton analyze = new JButton("Analyze");
        JButton upload = new JButton("Upload");

        JPanel panel = new JPanel();
        panel.setLayout(new FlowLayout());
        panel.add(analyze);
        panel.add(upload);

        frame.setLayout(new GridLayout(3,1));
        frame.add(inputField);
        JScrollPane inputScroll = new JScrollPane(inputField);
        frame.add(inputScroll);
        frame.add(panel);
        frame.add(outputField);
        JScrollPane outputScroll = new JScrollPane(outputField);
        frame.add(outputScroll);
        outputField.setEditable(false);

        analyze.addActionListener(e -> {
            outputField.setText("");
            RustLexer lexer = new
RustLexer(CharStreams.fromString(inputField.getText()));
            CommonTokenStream tokens = new CommonTokenStream(lexer);
            RustParser parser = new RustParser(tokens);

```



```

        ParseTree tree = parser.start();
        outputField.setText(tree.toStringTree(parser));
    });

    upload.addActionListener(new ActionListener() {
        @Override
        public
        void actionPerformed(ActionEvent e) {
            String initialPath = "D:\\University-Universytet\\42SI
Stvorennja IDE\\";
            JFileChooser chooser = new JFileChooser(initialPath);
            int returnedValue = chooser.showOpenDialog(frame);

            if (returnedValue == JFileChooser.APPROVE_OPTION) {
                File file = chooser.getSelectedFile();
                System.out.println(file.getPath());

                try {
                    FileReader reader = new FileReader(file);
                    BufferedReader bufferedReader = new
BufferedReader(reader);

                    String string1 = "";
                    StringBuilder string2 = new StringBuilder();

                    while ((string1 = bufferedReader.readLine()) != null)
{
                        string2.append(string1).append("\n");
                    }

                    inputField.setText(string2.toString());
                    bufferedReader.close();
                } catch (IOException fileNotFoundException) {
                    fileNotFoundException.printStackTrace();
                }
            }
        }
    });

    frame.setSize(400,300);
    frame.setVisible(true);
}
}

```

Вигляд

Вигляд програми наведено нижче на рисунках:

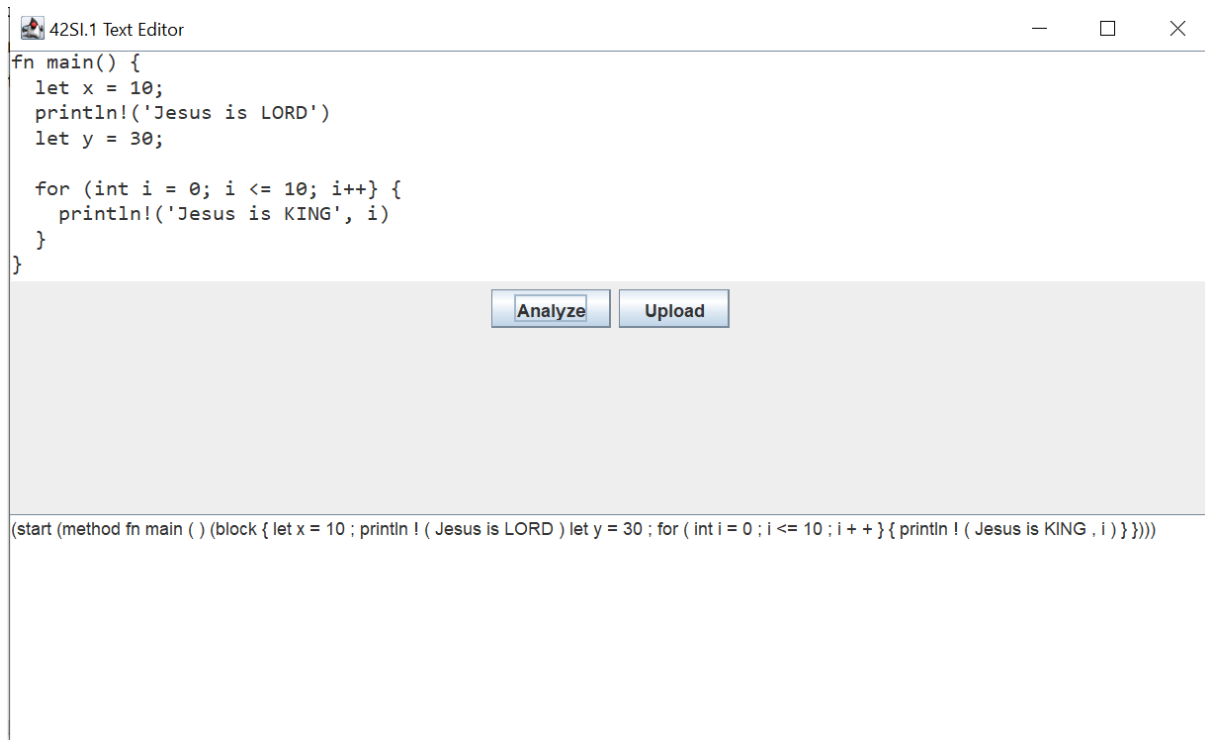


Рисунок 3.1 — Інтерфейс програми

4 Труднощі

Виникли наступні труднощі в процесі роботи:

- було додано елементи прокрутки у панель контейнер замість щоб додати їх до рамки;

ВИСНОВКИ

Благодаттю Господа Ісуса Христа зроблено роботу. Розроблено лексичний аналізатор мовою Antlr та програму мовою Java.

ПИТАННЯ

З чого складається сучасне інтегроване середовище розробки?

Складається з багатьох частин, зокрема:

- текстовий редактор;
- менеджер проєктів;
- інтеграція з системами контролю версій.

Чим відрізняються компілятори та інтерпретатори?

Компілятор перекладає весь код у машинний одразу, а інтерпретатор виконує його рядок за рядком.

Що є результатом та входом лексичного аналізатору?

Результатом лексичного аналізатору є потік токенів.